

University of Glasgow | School of Engineering

Digital Signal Processing

Bernd Porr & Nick Bailey

May 6, 2026

https://www.youtube.com/dspcourse

Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)

1 Acknowledgements

A big thanks to Fiona Baxter for typing up the handwritten lecture notes and turning them into LaTeX.

2 Introduction

This handout introduces you into the basic concepts of Digital Signal Processing. In contrast to the YouTube clips it focusses more on the theoretical aspects of it and its analytical derivations. The (video-) lectures cover more practical programming examples in Python and C++.

2.1 Suggested reading

- Digital Signal Processing by John G. Proakis and Dimitris K Manolakis
- Digital Signal Processing: System Analysis and Design by Paulo S. R. Diniz, Eduardo A. B. da Silva, and Sergio L. Netto

2.2 Advantages of digital signal processing

- flexible: reconfigurable in software!
- easy storage: numbers!
- cheaper than analogue design
- very reproducible results (virtually no drift)

3 Python

Python is a fully featured scripting language and it's very well thought through in its approach. It has a vast library of modules which means that only rarely you need implement low level functionality. It can be programmed both in a quick and dirty approach to try out new ideas and at the same time it supports object oriented programming so that truly professional programs can be developed.

Python has a straightforward interface to call C and C++ functions so that you can also mix it with C/C++ to speed up processing. For example a digital filter class might have the design parts written in python and the actual fast filter routines implemented in C. All this is supported by the python package management so that you can also deploy mixed python / C code without any problems. For example the AI library by google "tensorflow" can be installed as a python package and in the background it uses the GPUs on your computer without even noticing.

Here, we just comment on certain aspects of Python which are important for DSP. Check out the full documentation of python if you are interested beyond DSP in it. In particular it has also very easy to use support for GUI programming.

3.1 How to use Python?

There are two ways to use Python. The one is interactive within the python console and then the other method is by running a python script.

Development environments such as anaconda have both the console and then scripting facility integrated under one roof.

The python console The python console you can just start by typing ``python'':

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC
```

```
Type "help", "copyright", "credits" or "license"
>>> 3+4
7
>>>
```

then you can type in commands one by one. Under Windows you might need to point the PATH variable to your python program or you let Anaconda do it for you.

Python scripts Scripts are text files which contain python commands. One command per line and these are executed line by line. Write your commands in any text editor and save it as a .py-file. However, it's recommended to use an editor which supports at least python's indentation which is a very important feature of python. We'll see that indentation in python defines the body of a function, the content of a loop or the code executed conditionally. So, it's recommended for any serious work to install an editor which supports python such as spyder, vscode, pycharm or emacs. Once you have your script file saved just type ``python myscript.py'' and it will be executed.

Help / Tutorials There is an abundance of online help available for Python. The official Python pages have an excellent tutorial (https://docs.python.org/3/tutorial/) and of course you find answers to all your questions on Stack Overflow. If you need help for a function you can use the in built help function ``help(thecommand)'' to get help for ``thecommand''. Try out ``help(print)'' to get help for the ``print'' command.

3.2 Packages/Modules

The power of Python comes form thousands of modules which are shipped with python or can be installed via python's package manager. A package contains usually many modules. The standard package manager of python is called "pip" which downloads the package for you and installs it. This is not just very convenient but also is safe because the download location is the official python repository.

Packages can be installed system-wide, user-specific or within virtual environments. The problem is that if you install packages system wide that they might interfere with another user's package needs. A better solution is to install packages in the user's subdirectory (by using the `–user` option). However even that might cause problems because different python scripts might need different pip packages. For that reason now a single user can install packages into separate locations for different use cases which are called "virtual environments". The newest Ubuntu versions enforce these and you need to create at least one environment:

```
python –m venv myenv
```

which creates a subdirectory called `myenv`. If you want to use packages inside `myenv` you need to activate it with:

```
source ~/myenv/bin/activate
```

In general you import modules with the "import" command. For example "import numpy as np" imports the module numpy into python and abbreviates it as "np" so that you'd write "np.sin(c)" to call the sine function in numpy.

The import command loads essentially a file called "numpy.py" so that you can write your own modules just by creating file with that name.

Important modules for DSP are:

- numpy: Numerical operations for number crunching, matrix/array operations, trigonometric functions, etc. It offers fast numerical operations on arrays and matrices. It also has extensive Fourier Transform commands.
- pylab: Plotting functions, statistics and data fitting functions
- scipy: Scientific computing package. In particular we are insterested in scipy.signal which contains all the signal processing functions and filter design commands. It also offers Fourier Transforms, data I/O routines, vector quantisation, linear algebra, optimisation tools and statistics.

3.3 Plotting of Functions

Pylab has a vast amount of plotting functions which supports simple plotting with just a few commands up to complex animations of 3D data. For example, plotting a sine wave can be achieved with:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0,2*np.pi,100)
y = np.sin(x)
plt.plot(x,y)
plt.show() # only ONCE and at the end
```

where we have used the numpy `linspace` command to create an array which runs from 0 to two pi with 100 samples.

Important: There is only a single `plt.show()` at the very end of your program. This triggers the actual display of all plot-windows and then blocks.

If you want to create more plot-windows you can use the command `plt.figure`. Access the windows with the commands `plt.figure(1)`, `plt.figure(2)`.

It is also possible to combine plot windows into one with the command `subplot`.

How to make it look nice? Here is an example:

```
plt.xlabel('time (s)')
plt.ylabel('voltage (V)')
plt.title('That is an AC voltage over time!')
plt.grid(True)
```

Generally the best approach is to use an example and then customise it to your needs.

3.4 Importing/Exporting data

Space/Tab/Comma separated data Numpy has a very handy import function for data as long as the the elements in the data file are separated with spaces, tabs or commas. To load such data files just type in:

```
x = np.loadtxt('mydatafile.dat')
```

This creates a one- or two-dimensional array which reflects exactly the structure in the datafile. You can also save such numpy arrays again with:

```
numpy.savetxt("foo.csv", a, delimiter="\t") # ta
```

Thus, if you want to write software which creates Python readable data then just export the data as space/tab/comma separated ASCII files. Generally for readability of the textfiles tab-separated is recommended.

WAV files Scipy contains in its submodule io a subsubmodule wavfile which supports reading and writing wav files:

```
import scipy.io.wavfile as wavfile
fs,data = wavfile.read('kongas.wav');
```

fs,data is a tuple containing the sampling rate and then the data.

```
import scipy.io.wavfile as wavfile
wavfile.write('kongas_filt.wav',fs,y3);
```

whereas 'fs' is the sampling rate and y3 the data -- usually normalised between -1 and +1 for floating point numbers.

4 Signal conversion

We start with a few definitions when we transform an analogue signal into the digital domain and back:

1. **Sampling**

$$\underbrace{x_a(nT)}_{\text{analogue signal}} \quad \equiv \quad \underbrace{x(n)}_{\text{discrete data}} \tag{1}$$

   - $x_a(t)$: analogue signal
   - T: sampling interval, $\frac{1}{T} = F_s$ is the sampling rate

- $x(n)$: discrete data

2. **Quantisation** Continuous value into discrete steps. $x(n) \rightarrow x_q(n)$
3. **Coding** $x_q(n) \rightarrow$ into binary sequence or integer numbers.

4.1 Sampling

The analogue signal X_a is sampled at intervals T to get the sampled signal $X(n)$.

$$x(n) = x_a(nT), \qquad -\infty < n < +\infty \tag{2}$$

Normalised frequency

Note that the sampling frequency

$$F_s = \frac{1}{T_s} \tag{3}$$

is lost and needs to be stored additionally so that the analogue signal can be reconstructed:

$$x_a(t) = x_a(nT) = x_a(\frac{n}{F_s}) \tag{4}$$

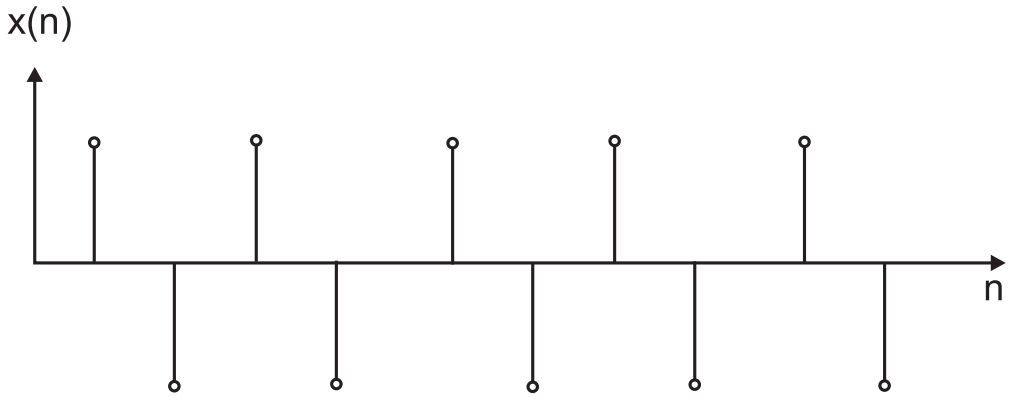


Figure 1 Maximum frequency of a sampled signal is half the normalised frequency.

What is the frequency range of our digital signal? What is the maximum frequency in digital terms? Fig 1 shows that the max frequency is 0.5 because we need two samples to represent a "wave". The frequency range $0 \dots 0.5$ is called the *normalised frequency* range. What is the relation between normalised frequency and sampling rate?

Let's have a look at the analog signal $X_a(t)$ with the frequency F:

$$x_a(t) = A\cos(2\pi F t) \tag{5}$$

and we sample it only at:

$$t = n/F_s \tag{6}$$

Then the digital signal is:

$$x_a(n) = A\cos(2\pi F/F_s n) \tag{7}$$

Now, we can define the normalised frequency as:

$$\text{Normalised frequency:} \qquad f = \frac{F}{F_s} \tag{8}$$

which has its max value at 0.5 which represents one period of a sine wave within two samples [1]

Nyquist frequency

Recall that the max value for the normalised frequency f is 0.5 and that:

$$x_a(n) = A\cos(2\pi n f) \tag{9}$$

with n as an integer because we are sampling.

What happens above $f>0.5$? Imagine $f = 1$

$$x_a(n) = \cos(2\pi n) = 1 \qquad f = 1 \tag{10}$$

which gives us DC or zero frequency. The same is true for $f = 2, 3, 4, \ldots$. We see that above $f = 0.5$ we never get higher frequencies. Instead they will always stay between $0 \ldots 0.5$ for the simple reason that it is not possible to represent higher frequencies. This will be discussed later in greater detail.

The ratio F/F_s must be lower than 0.5 to avoid ambiguity or in other words the maximum frequency in a signal must be lower than $\frac{1}{2}F_s$. This is the Nyquist frequency.

If there are higher frequencies in the signal then these frequencies are “folded down” into the frequency range of $0 \dots \frac{1}{2}F_s$ and creating an alias of its original frequency in the so called “baseband” ($f = 0 \dots 0.5$). As long as the alias is not overlapping with other signal components in the baseband this can be used to downmix a signal. This leads to the general definition of the sampling theorem which states that the bandwidth B of the input signal must be half of the sampling rate F_s:

$$B < \frac{1}{2}F_s \tag{11}$$

The frequency $\frac{1}{2}F_s$ is called the Nyquist frequency.

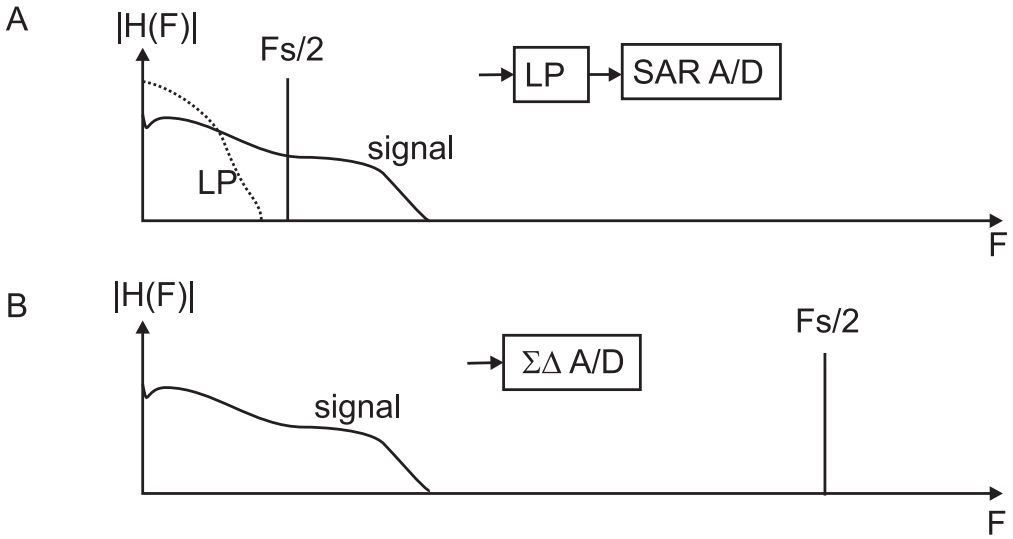


Figure 2 Anti alias filtering. A) with a lowpass filter. B) with a sigma delta converter with very high sampling rate.

What do we do if the signal contains frequencies above $\frac{1}{2}F_s$? There are two ways to tackle this problem: The classical way is to use a lowpass filter (see Fig. 2A) which filters out all frequencies above the Nyquist frequency. However this might be difficult in applications with high resolution A/D converters. Alternatively one can use a much higher sampling rate to avoid aliasing. This is the idea of the sigma delta converter which operates at sampling rates hundred times higher than the Nyquist frequency.

Sampling theorem

Is it possible to reconstruct an analogue signal from a digital signal which contains only frequencies below the Nyquist frequency?

$$F_s > 2F_{\max} \tag{12}$$

where $F_{\max}$ is max frequency in the signal which we represent by sine waves:

$$x_a(t) = \sum_{i=1}^{n} A_i \cos(2\pi F_i t + \Theta_i) \tag{13}$$

The analogue signal $x_a(t)$ can be completely reconstructed if:

$$g(t) = \frac{\sin 2\pi Bt}{2\pi Bt} \tag{14}$$

with

$$B = F_{\max} \tag{15}$$

$$x_a(t) = \sum_{h=-a}^{a} x_a(\frac{n}{F_s}) g(t - \frac{h}{F_s}) \tag{16}$$

The problem is that $g(t)$ runs from negative time to positive time and as we see later is a-causal so that this cannot be implemented for real but approximations of $g(t)$ are possible and are analogue lowpass filters which smooth out the step like outout of an digital to analogue converter.

4.2 Quantisation

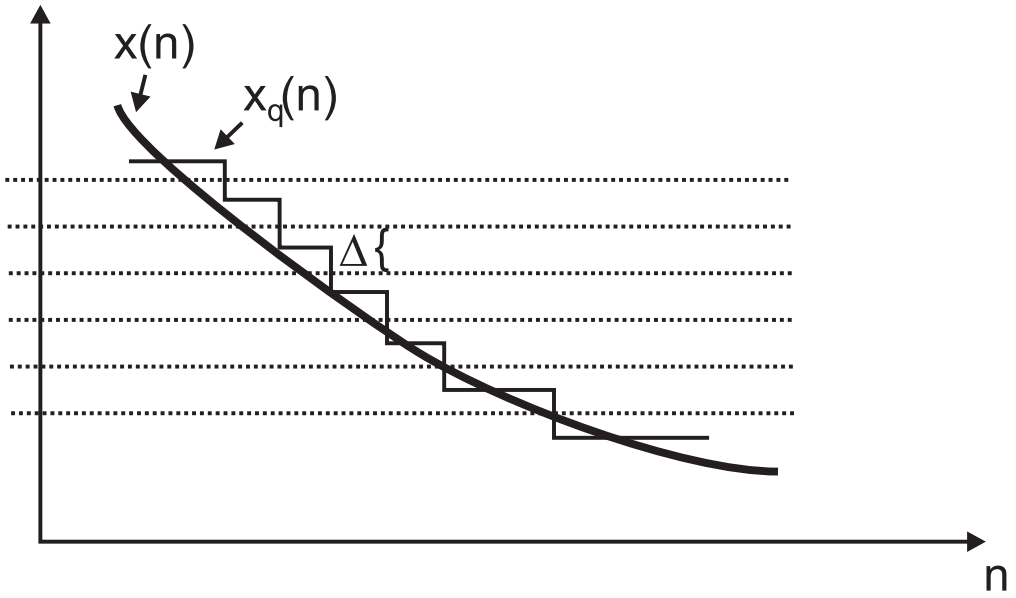


Figure 3 Δ is the quantisation step.

A/D converters have a certain resolution. For example, the MAX1271 has a resolution of 12 bits which means that it divides the input range into 4096 equal steps (see Fig 3).

$$\Delta = \text{quantisation step} = \frac{x_{\max} - x_{\min}}{L-1} \qquad (17)$$

where $x_{\max} - x_{\min}$ is the dynamic range in volt (for example $4.096V$) and L is the number of quantisation steps (for example, 4096). Δ is the quantisation step which defines minimal voltage change which is needed to see a change in the output of the quantiser. The operation of the quantiser can be written down as:

$$x_q(n) = Q[x(n)] \tag{18}$$

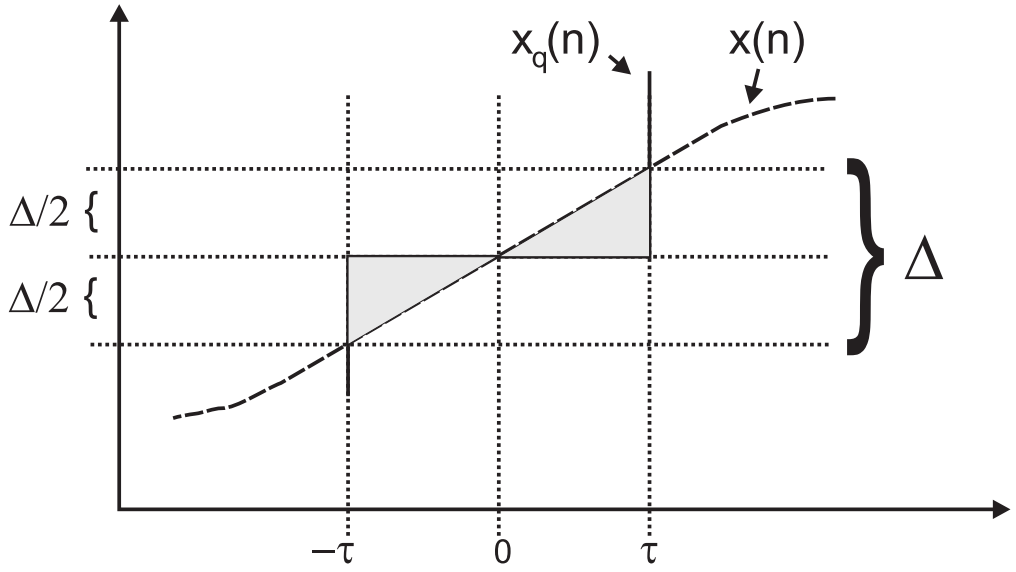


Figure 4 Illustration of the quantisation error. It is zero at $t = 0$ and increases to the edges of the sampling interval. Illustrated is the worst case scenario. This repeats in the next sampling interval and so forth.

Fig. 4 shows error produced by the quantiser in the worst case scenario. From that it can be seen that the maximum quantisation error is half the quantisation step:

$$-\frac{\Delta}{2} \leq e(n) \leq \frac{\Delta}{2} \tag{19}$$

The smaller the quantisation step Δ the lower the error!

What is the mean square error P_q?

$$P_q = \frac{1}{\tau}\int_0^\tau e_q^2(t)dt \tag{20}$$

$$P_q = \frac{1}{\tau}\int_0^\tau \left(\frac{\Delta}{2\tau}\right)^2 t^2 dt \tag{21}$$

$$= \frac{\Delta^2}{4\tau^3}\int_0^\tau t^2 dt \tag{22}$$

$$P_q = \frac{\Delta^2}{12\tau^3}\tau^3 = \frac{\Delta^2}{12} \tag{23}$$

this then results in the almost trival result that the size of the quantisation step Δ scales linarly with the average error $\sqrt{P_q}$. So if we have two times more quantisation steps then the error will half!

What is the relative error to a sine wave?

$$P_x = \frac{1}{T_p}\int_0^{T_p} (A\cos\Omega t)^2 dt = \frac{A^2}{2} \tag{24}$$

Ratio to signal power to noise:

$$\mathrm{SQNR} = \frac{P_x}{P_q} = \frac{A^2}{2}\cdot\frac{12}{\Delta^2} \tag{25}$$

$$= \frac{6A^2}{\Delta^2} \tag{26}$$

This equation needs to be interpreted with care because increasing the amplitude of the input signal might lead to saturation if the input range of the A/D converter is exceeded.

5 Frequency representation of signals

Often we are more interested in the frequency representation of signals than the evolution in time.

We will use small letters for time domain representation and capital letters for the frequency representation, for example $X(F)$ and $x(t)$.

5.1 Continuous time and frequency

Periodic signals

Periodic signals can be composed of sine waves :

$$x(t) = \sum_{k=-\infty}^{\infty} c_k e^{j2\pi k F_1 t} \tag{27}$$

where F_1 is the principle or fundamental frequency and $k \neq 1$ are the harmonics with strength c_k. Usually c_1 is set to 1. This is a Fourier series with c_k as coefficients. For example an ECG has a fundamental frequency of about 1Hz (60 beats per minute). However, the harmonics give the ECG its characteristic peak like shape.

How do we find the coefficients c_k?

$$c_k = \frac{1}{T_p} \int_{T_p} x(t) e^{-j2\pi k F_1 t} dt \tag{28}$$

For simple cases there are analytical solutions for c_k, for example for square waves, triangle wave, etc.

What are the properties of c_k?

$$c_k = c_{-k}^* \qquad \Leftrightarrow \qquad x(t) \text{ is real} \tag{29}$$

or

$$c_k = \quad | \, c_k \, | \, e^{j\theta_k} \tag{30}$$
$$c_{-k} = \quad | \, c_k \, | \, e^{-j\theta_k} \tag{31}$$

Proof: with the handy equation…

$$\cos z = \frac{1}{2} \left(e^{zi} + e^{-zi} \right) \tag{32}$$

we get

$$x(t) = \quad c_0 + \sum_{k=1}^{\infty} | \, c_k \, | \, e^{j\theta_k} e^{j2\pi k F_1 t} + \sum_{k=1}^{\infty} | \, c_k \, | \, e^{-j\theta_k} e$$
$$= \quad c_0 + 2 \sum_{k=1}^{\infty} | \, c_k \, | \cos(2\pi k F$$

How are the frequencies distributed? Let's have a look at the frequency spectrum of a periodic signal: $P_k = | \, c_k \, |^2$

- There are discrete frequency peaks
- Spacing of the peaks is $\frac{1}{T_1} = F_1$
- Only the positive frequencies are needed: $c_{-k} = c_k^*$

A-periodic signals

In case nothing is known about $X(t)$ we need to integrate over all frequencies instead of just the discrete frequencies.

$$X(F) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi Ft} dt \tag{35}$$

Consequently, the frequency spectrum $X(F)$ is continuous.

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(F) e^{j2\pi Ft} dF \tag{36}$$

5.2 Sampled time and/or frequency

Discrete time Fourier Transform (DTFT)

The signal $x(n)$ is discrete whereas the resulting frequency spectrum is considered as continuous (arbitrary signals).

- Analysis or direct transform:

$$X(\omega) = \sum_{n=-\infty}^{\infty} x(n) e^{-j\omega n} \tag{37}$$

- Synthesis or inverse transform:

$$x(n) = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(\omega) e^{j\omega n} d\omega \tag{38}$$

$$= \frac{1}{2\pi} \int_{-0.5}^{0.5} X(f) e^{j2\pi fn} df \tag{39}$$

note the range here. f is the normalised frequency.

The effect of time domain sampling on the spectrum (Sampling theorem)

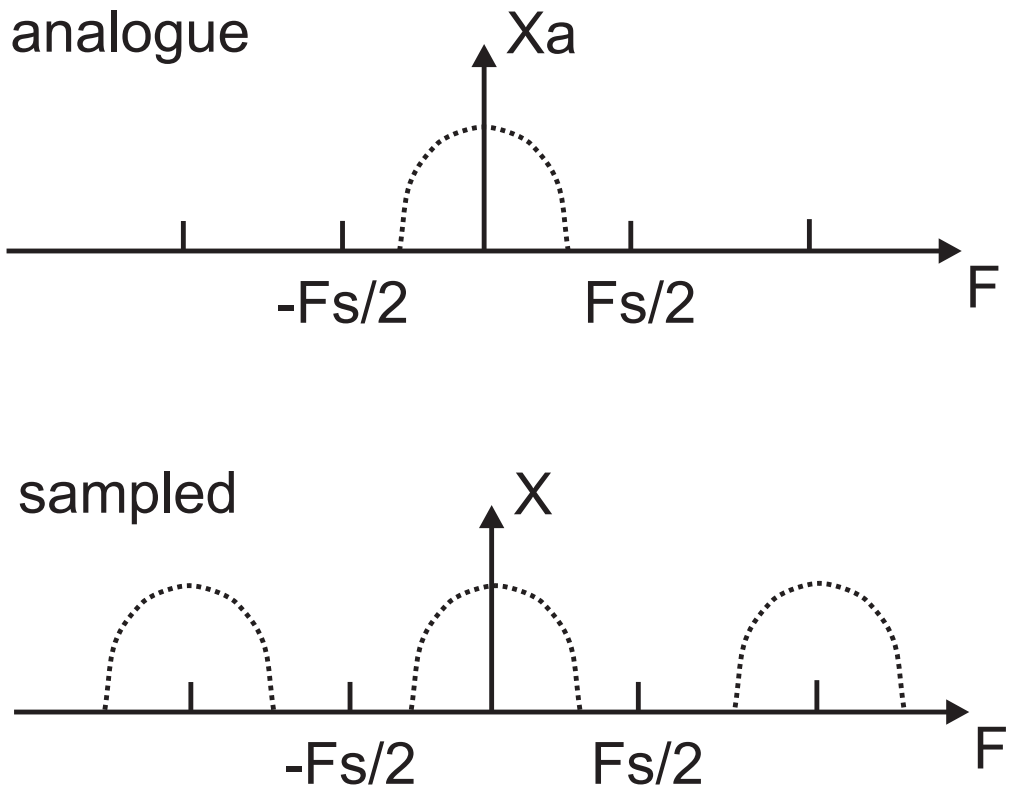


Figure 5 Effect of sampling on the spectrum.

What effect has time domain sampling on the frequency spectrum [2] ? Imagine we have an analog spectrum $X_a(F)$ from a continuous signal $x(t)$. We want to know how the spectrum $X(F)$ of the discrete signal $x(n)$ looks like.

$$X(F) \Leftrightarrow X_a(F) \tag{40}$$

The signal is represented by $x(n)$ so that we can equate the Fourier transforms of the sampled spectrum $X(F)$ and of the analogue spectrum $X_a(F)$.

$$\int_{-0.5}^{0.5} \underbrace{X(f)}_{\text{sampled}} e^{j2\pi fn} df = \int_{-\infty}^{+\infty} \underbrace{X_a(F)}_{\text{cont}} e^{j2\pi nF/F_s} dF \quad ($$

Obviously $X(f)$ must be different to accommodate the different integration ranges. The trick is now to divide the integral on the right hand side of Eq. 41 into chunks to make it compatible to the range on the left hand hand side.

Remember that the normalised frequency is $f = F/F_s$ which allows us to change the integration to analogue frequency on both sides:

$$\frac{1}{F_s} \int_{-F_s/2}^{F_s/2} X(\frac{F}{F_s}) e^{j2\pi nF/F_s} dF = \int_{-\infty}^{+\infty} X_a(F) e^{j2\pi nF/F_s}$$

and now we divide the right hand side into chunks of F_s which corresponds to the integration range on the left hand side.

$$\begin{aligned}
\int_{-\infty}^{+\infty} X_a(F) e^{j2\pi n \frac{F}{F_s}} dF &= \sum_{k=-\infty}^{\infty} \int_{-\frac{1}{2}F_s+kF_s}^{+\frac{1}{2}F_s+kF_s} X \\
&= \sum_{k=-\infty}^{\infty} \int_{-\frac{1}{2}F_s}^{+\frac{1}{2}F_s} X_a(F-kF \\
&= \int_{-\frac{1}{2}F_s}^{+\frac{1}{2}F_s} \underbrace{\sum_{k=-\infty}^{\infty} X_a(F-}_{=X(F) \text{ of Eq.}}
\end{aligned}$$

This gives us now an equation for the sampled spectrum:

$$X(F/F_s) = \qquad F_s \sum_{k=-\infty}^{\infty} X_a(F - kF_s) \qquad (46)$$

$$X(f) = \qquad F_s \sum_{k=-\infty}^{\infty} X_a[(f - k)F_s] \qquad (47)$$

This equation can now be interpreted. In order to get the sampled spectrum $X(F)$ we need to make copies of the analog spectrum $X_a(F)$ and place these copies at multiples of the sampling rate F_s (see Fig. 5). This illustrates also the *sampling theorem*: if the bandwidth of the spectrum is wider than $F_s/2$ then the copies of the analogue spectrum will overlap and reconstruction would be impossible. This is called aliasing. Note that it is not necessary bad that the spectrum of the analogue signal lies within the range of the so called "base band" $-F/2 \ldots F/2$. It can also lie in another frequency range further up, for example $-F/2 + 34 \ldots F/2 + 34$ as long as the *bandwidth* does not exceed $F_s/2$. If it is placed further up it will automatically show up in the baseband $-F/2 \ldots F/2$ which is called "fold down". This can be used for our purposes if we want to down mix a signal.

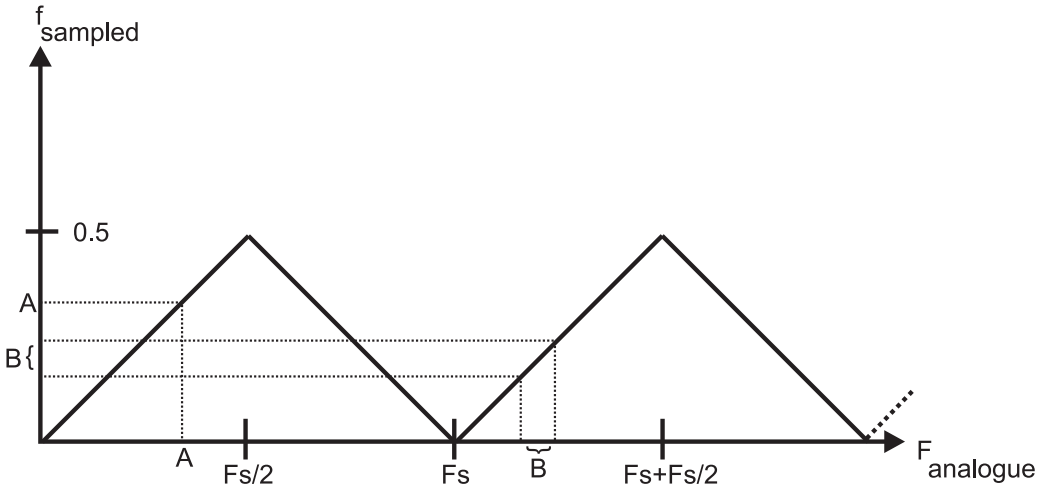


Figure 6 Mapping of frequencies in the analogue domain (F) and sampled domain (f). F_s is the sampling rate.

With the insight from these equations we can create a plot of how analogue frequencies map onto sampled frequencies. Fig 6 shows how the analogue frequencies F_{analogue} map on the normalised frequencies f_{sampled}. As long as the analogue frequencies are below $F_s/2$ the mapping is as usual as shown in Fig 6A. Between $F_s/2$ and F_s we have an inverse mapping: an increase in analogue frequency causes a decrease in frequencies. Then, from F_s we have again an increase in frequencies starting from DC. So, in general if we keep a bandlimited signal within one of these slopes (for example from F_s ... $F_s + 1/2F_s$ as shown in Fig 6B) then we can reproduce the signal.

This leads us to the generalised Nyquist theorem: if a bandpass filtered signal has a bandwidth of B then the minimum sampling frequency is $F_s = 2B$.

Discrete Fourier Transform (DFT)

So far we have only sampled in the time domain. However, on a digital computer the Fourier spectrum will always be a discrete spectrum.

The discrete Fourier Transform (DFT) is defined as:

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N} \qquad k = 0, 1, 2, \dots, N-1 \quad ($$

where N is the number of samples in both the time and frequency domain.

The inverse discrete Fourier Transform (IDFT) is defined as:

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N} \qquad n = 0, 1, 2, \dots, N-1$$

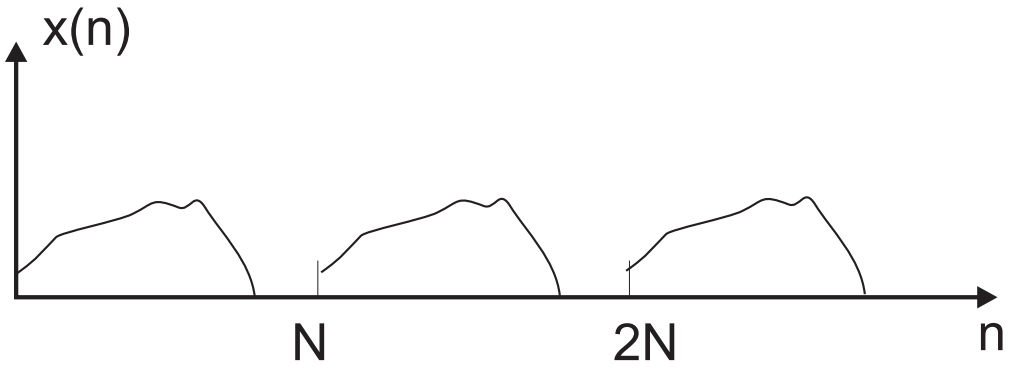


Figure 7 Effect of sampling in the frequency domain. The inverse in the time domain $x(n)$ contains repetitions of the original signal.

What is the effect in the timedomain of this discretisation [3] ? We start with the continuous Fourier transform and discretise it into N samples in the frequency domain:

$$X\left(\frac{2\pi}{N}k\right) = \sum_{n=-\infty}^{\infty} x(n) e^{-j\frac{2\pi}{N}kn} \qquad k = 0, \dots, N-1$$

Let's subdivide the sum into chunks of length N:

$$X\left(\frac{2\pi}{N}k\right) = \sum_{l=-\infty}^{\infty} \sum_{n=lN}^{lN+N-1} x(n) e^{-j\frac{2\pi}{N}kn} \quad (51$$

$$= \sum_{l=-\infty}^{\infty} \sum_{n=0}^{N-1} x(n-lN) e^{-j\frac{2\pi}{N}kn} \quad (52$$

$$= \sum_{n=0}^{N-1} \underbrace{\sum_{l=-\infty}^{\infty} x(n-lN)}_{\text{Periodic repetition!}} e^{-j\frac{2\pi}{N}kn} \quad (53$$

We note the following:

- Ambiguity in the time domain
- The signal is repeated every N samples

Practically this repetition won't show up because the number of samples is limited to N in the inverse transform. However, for operations which shift signals in the frequency domain it is important to remember that we shift a periodic time series. If we shift it out at the end of the array we will get it back at the start of the array.

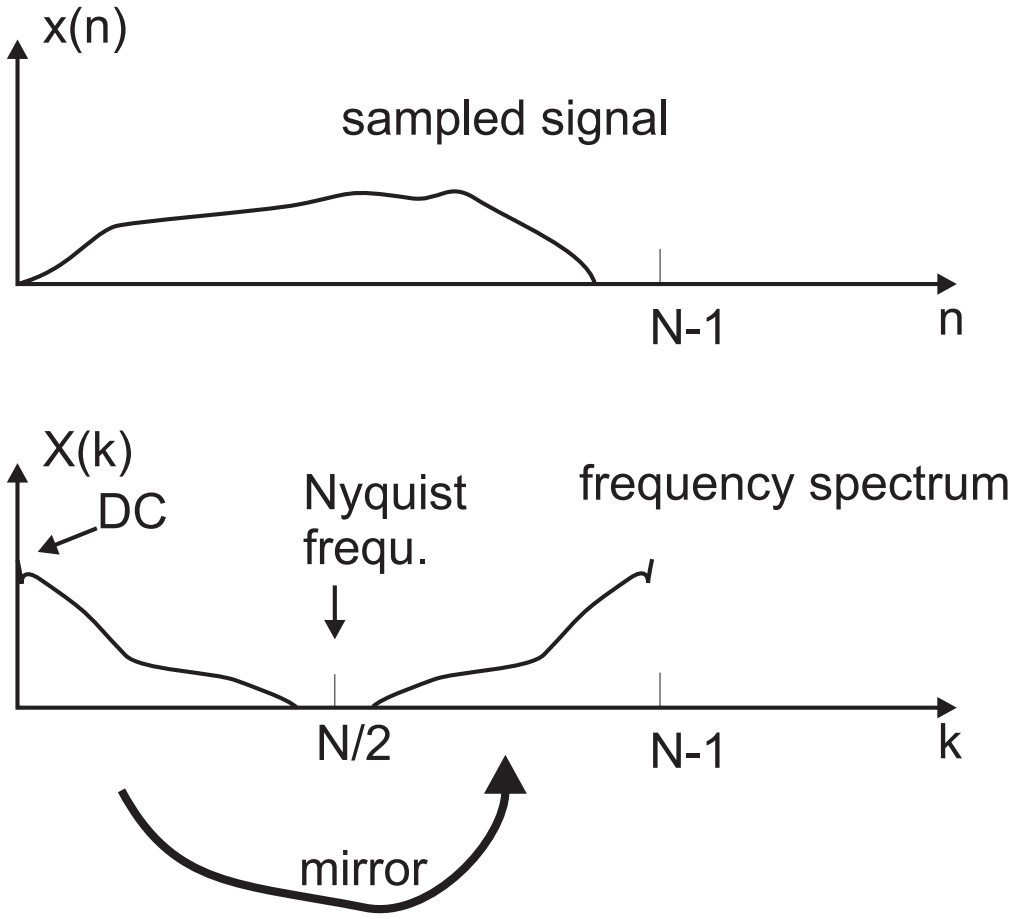


Figure 8 Example of a DFT. The sampled signal $x(n)$ is converted to the spectrum $X(k)$. Both have N samples. Note the redundancy: the spectrum is mirrored around $N/2$. The first value $X(0)$ is the DC value of the signal.

However this is the only sample which is not mirrored.

Fig. 8 shows an example of a DFT. It is important to know that the spectrum is mirrored around $N/2$. DC is represented by $X(0)$ and the Nyquist frequency $F_s/2$ is represented by $X(N/2)$. The mirroring occurs because the input signal $x(n)$ is *real*. This is important if one wants to modify the spectrum $X(F)$ by hand, for example to eliminate 50Hz noise. One needs to zero two elements of $X(F)$ to zero. This is illustrated in this python code:

```
import scipy as sp
yf=sp.fft(y)
# the sampling rate is 1kHz. We've got 2000 samp
# midpoint at the ifft is 1000 which corresponds
# So, 100 corresponds to 50Hz
yf[99:101+1]=0;
# and the mirror
yf[1899:1901+1]=0;
#
yi=sp.ifft(yf);
```

This filters out the 50Hz hum from the signal y with sampling rate 1000Hz. The signal `yi` should be real valued again or contain only very small complex numbers due to numerical errors.

Properties of the DFT

- Periodicity:

$$x(n+N) = x(n) \quad (54)$$
$$X(k+N) = X(k) \quad (55)$$

- Symmetry: if $x(n)$ real:

$$x(n) \text{ is real} \Leftrightarrow X^*(k) = X(-k) = X(N-k) \quad (5$$

This is important when manipulating $X(k)$ by hand.

- Time Reversal:

$$x(n) \leftrightarrow \quad X(k) \tag{57}$$
$$x(-n) \leftrightarrow \quad X(N-k) \tag{58}$$

- *Circular* convolution:

$$X_1(k)X_2(k) \leftrightarrow x_1(n) * x_2(n) \tag{59}$$

with

$$x_3(m) = \sum_{n=0}^{N-1} x_1(n)x_2(m-n) \tag{60}$$

More useful equations are at Proakis and Manolakis (1996 , pp.415) .

Problems with finite length DFTs

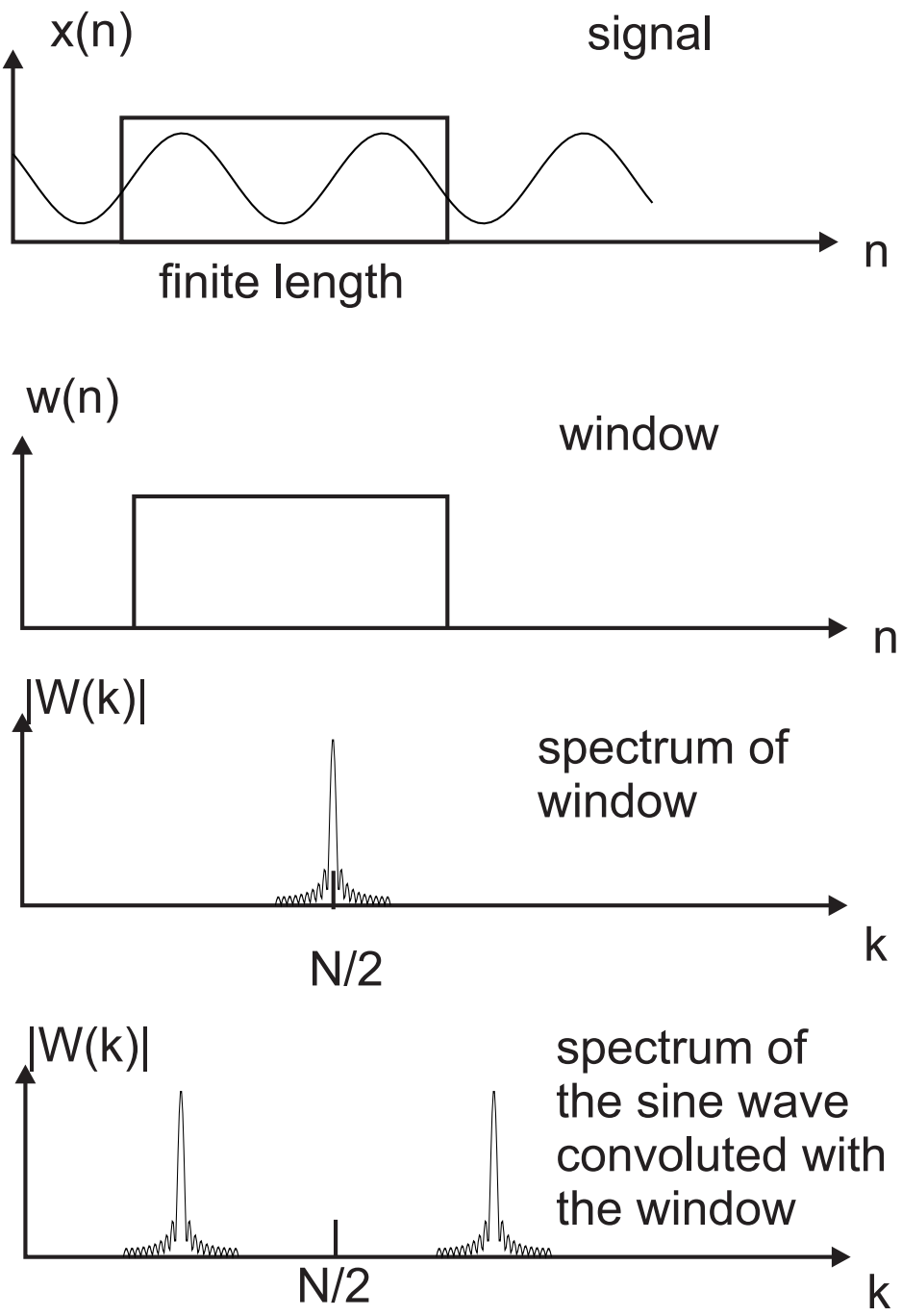


Figure 9 The effect of windowing on the DFT.

$$x_w(n) = x(n) \cdot w(n) \tag{61}$$

where x is the input signal and $w(n)$ represents a function which is 1 from $n = 0$ to $n = L - 1$ so that we have L samples from the signal $x(n)$.

To illustrate the effect of this finite sequence we introduce a sine wave $x(n) = \cos \omega_0 n$ which has just two peaks in a proper Fourier spectrum at $-\omega$ and $+\omega$.

The spectrum of the rectangular window with the width L is:

$$W(\omega) = \frac{\sin(\omega L/2)}{\sin(\omega/2)} e^{-j\omega(L-1)/2} \tag{62}$$

The resulting spectrum is then:

$$X_3(\omega) = X(\omega) * W(\omega) \tag{63}$$

Because the spectrum of $X(\omega)$ consists of just two delta functions the spectrum $X_3(\omega)$ contains the window spectrum $W(\omega)$ twice at $-\omega$ and $+\omega$ (see Fig. 9). This is called *leakage*. Solutions to solve the leakage problem? Use Windows with a narrower spectrum and with less ripples (see FIR filters).

Fast Fourier Transform

We can rewrite the DFT (Eq. 48) in a slightly more compact form:

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \tag{64}$$

with the constant:

$$W_N = e^{-j2\pi/N} \tag{65}$$

The problem with the DFT is that it needs N^2 multiplications. How can we reduce the number of multiplications? Idea: Let's divide the DFT in an odd and an even sequence:

$$x(2m) \tag{66}$$

$$x(2m+1), \qquad m = 0, ..., \frac{N}{2} - 1 \tag{67}$$

which gives us with the trick $W_N^{2mk} = W_{N/2}^{mk}$ because of the definition Eq. 65.

$$\begin{aligned} X(k) = & \sum_{m=0}^{N/2-1} x(2m) W_N^{2mk} + \sum_{m=0}^{N/2-1} x(2m+1) W_N^{k(} \\ = & \sum_{m=0}^{N/2-1} x(2m) W_{N/2}^{mk} + W_N^k \sum_{m=0}^{N/2-1} x(2m+1) \\ = & F_e(k) + W_N^k \end{aligned}$$

F_e and F_o have both half the length of the original sequence and need only $(N/2)^2$ multiplication, so in total $2 \cdot (N/2)^2 = \frac{N^2}{2}$. Basically by dividing the sequence in even and odd parts we can reduce the number of multiplications by 2. Obviously, the next step is then to subdivide the two sequences $F_e(k)$ and $F_o(k)$ even further into something like $F_{ee}(k), F_{eo}(k), F_{oe}(k)$ and $F_{oo}(k)$.

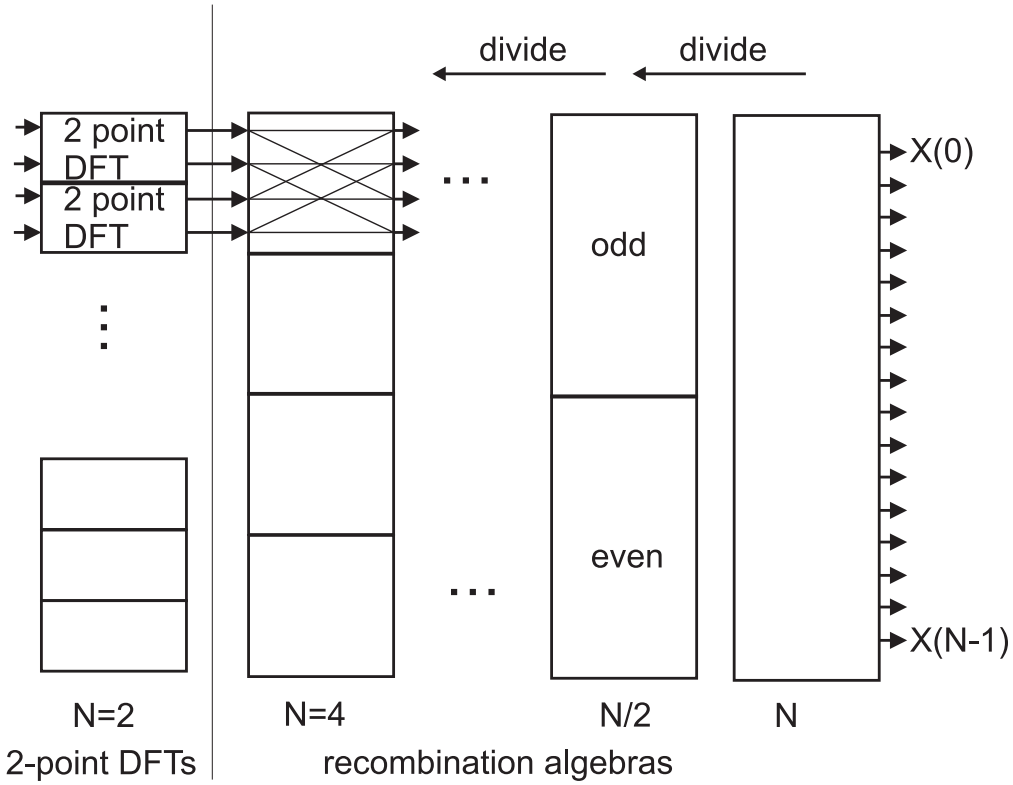


Figure 10 Illustration of the Fast Fourier Algorithm. The sequence of N samples is recursively divided into subsequences of odd and even samples.

In general the recipe for the calculation of the FFT is:

$$X_i(k) = X_{ie}(k) + W_L^k X_{io}(k) \tag{71}$$

W_L^k is the phase factor in front of the odd sequence. This is continued until we have only two point ($N = 2$) DFTs (see Eq. 64):

$$\text{DC:} \qquad X(0) = \qquad x(0) + \underbrace{W_2^0}_{1} x(1) = x($$

$$\text{Nyquist frequ.:} \qquad X(1) = \qquad x(0) + \underbrace{W_2^1}_{-1} x(1) = x($$

A two point DFT operates only on two samples which can represent only two frequencies: DC and the Nyquist frequency which makes the calculation trivial. Eq. 72 is an averager and Eq. 73 is basically a differentiator which gives the max output for the sequence $1, -1, 1, -1, ...$ Fig. 10 illustrates how to divide the initial sequence to arrive at 2 point DFTs. In other words the calculation of the full DFT is done by first calculating $N/2$ 2 point DFTs and recombining the results with the help of Eq. 71. This is sometimes called the “Butterfly” algorithm because the data flow diagram can be drawn as a butterfly. The number of complex multiplications reduces in this approach to $N \log_2 N$ which is actually the worst case scenario because many W_L^k usually turn out to be $1, -1, j, -j$ which are just sign inversions or swaps of real and imaginary parts. A clever implementation of this algorithm will be even faster.

In summary the idea behind the FFT algorithms is to divide the sequence into subsequences. Here we have presented the most popular radix 2 algorithm. The radix 4 is even more efficient and there are also algorithms for divisions into prime numbers and other rather exotic divisions. However, the main idea is always the same: subsample the data in a clever way so that the final DFT becomes trivial.

5.3 Software

In Teukolsky et al. (2007) you’ll find highly optimised C code for Fourier transforms. Most Linux distros (Ubuntu, Suse, RedHat, …) come with an excellent FFT library called `libfftw3`.

5.4 Visualisation

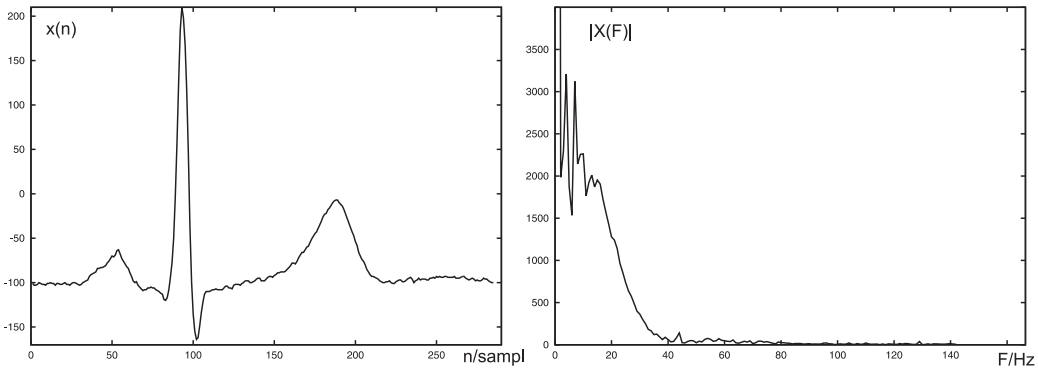


Figure 11 An ECG signal in the time- and frequency domain.

The Fourier spectrum is a complex function of frequency which cannot be displayed in a convenient way. Usually the amplitude or magnitude of the spectrum is of interest (see Fig. 11) which is calculated as the absolute value $|X(F)|$ of the Fourier spectrum X(F). In addition the phase might be of interest which is calculated as $\arg(X(F))$.

- Magnitude or Amplitude: $|X(F)|$
- Phase: $\arg(X(F))$

6 Causal Signal Processing

In realtime systems and any system where data is processed as it arrives the values of the future samples are not known. This is as in an analogue system where we send for example a signal into an amplifier and it needs to process it as it arrives. The amplifier won't know if the next music track is Mozart or Jimmy Hendrix. It just needs to react to what it's fed into it.

However, the Fourier Transform is *not real time*. We need the whole signal from the first to the last sample. For that reason we need to develop a new mathematical framework which we call “Causal Signal Processing”. These systems should ideally react to an incoming signal as an analogue system does, namely as fast as possible with little latency as possible.

In the next section we are now developing a mathematical framework for causal digital signal processing. We draw here heavily from analoge circuit design as this is by its own nature performs realtime causal processing.

6.1 Causality

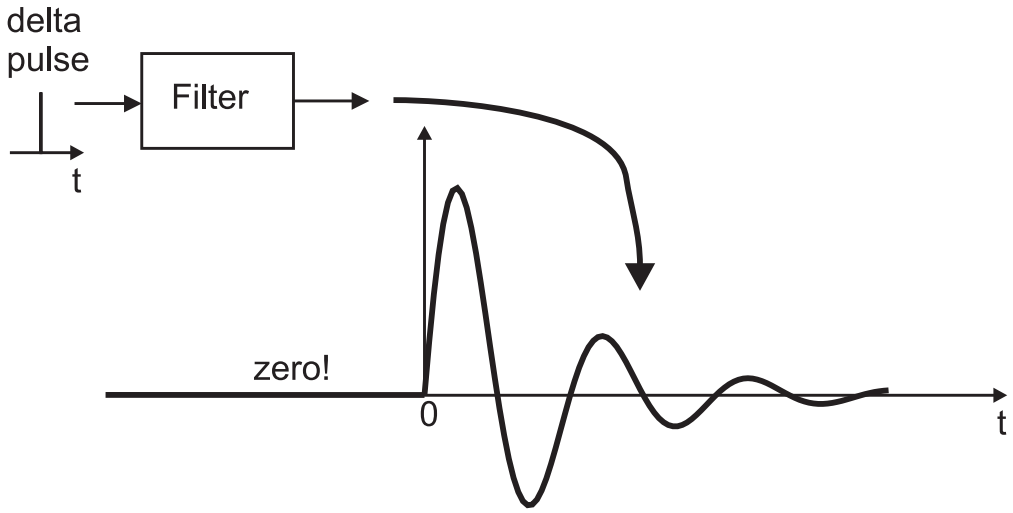


Figure 12 A causal system only *reacts* to it’s input. Causal signals only evolve in positive time. Per definition the signal is zero for $t<0$.

Fig. 12 illustrates the concept of causality. Causal systems cannot look into the future. They can only react to a certain input. Causal signals are kept zero for $t<0$ per definition.

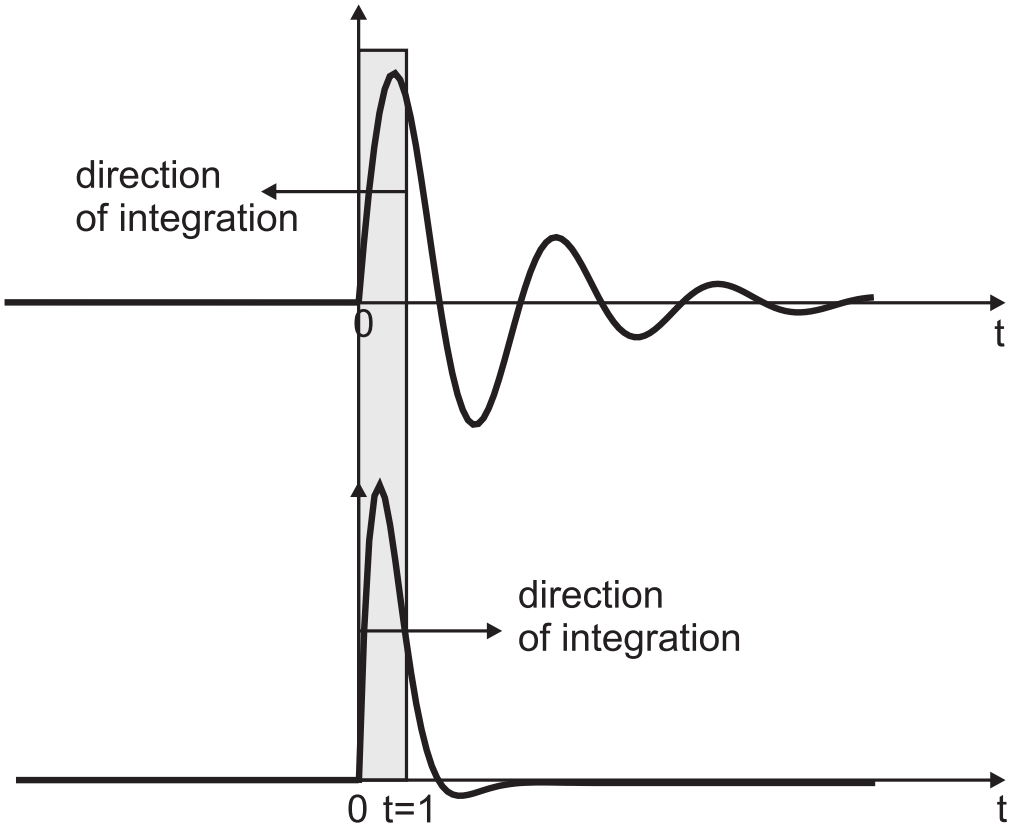


Figure 13 Illustration of the convolution. The shaded area shows the integration for $t = 1$.

6.2 Convolution of Causal Signals

After having defined causality we can define the convolution in continous time:

$$y(t) = \quad h(t) * x(t) = \int_{-\infty}^{\infty} h(t-\tau)x(\tau)d\tau \quad (74)$$

$$y(n) = \quad h(n) * x(n) = \sum_{n=-\infty}^{\infty} h(n)x(m-n) \quad (75)$$

Note the reversal of integration of the function h. This is characteristic of the convolution. At time $t = 0$ only the values at $h(0)$ and $x(0)$ are evaluated (see Fig. 13). Note that both functions are zero for $t<0$ (causality!). At $t>0$ the surface which is integrated grows as shown in Fig. 13 for $t = 1$.

What happens if $x(t) = \delta(t)$? Then Eq. 75:

$$y(t) = \int_{-\infty}^{\infty} h(t-\tau)\delta(\tau)d\tau = h(t) \tag{76}$$

provides us with the function $h(t)$ itself. This will be used later to determine the impulse response of the filter.

6.3 Laplace transform

The Fourier transform is not suitable for causal systems because it requires the whole signal from $-\infty<t<+\infty$. What we need is a transform which works with continuous causal signals. This is the Laplace transform:

$$\mathbf{LT}\left(h(t)\right) = H(s) = \int_{0}^{\infty} h(t)e^{-st}dt \tag{77}$$

The Laplace transform has a couple of very useful properties:

- Integration:

$$\int f(\tau)d\tau \Leftrightarrow \frac{1}{s}F(s) \tag{78}$$

- Differentiation:

$$\frac{d}{dt}f(t) \Leftrightarrow sF(s) \tag{79}$$

- Shift:

$$f(t-T) \Leftrightarrow e^{-Ts}F(s) \tag{80}$$

Proof:

$$\begin{aligned}
\mathbf{LT}\left(h(t-T)\right) = & \int_0^\infty h\underbrace{(t-T)}_{causal}e^{-st}dt & (81)\\
= & \int_0^\infty h(t)e^{-s(t+T)}dt & (82)\\
= & \int_0^\infty h(t)e^{-st}e^{-sT}dt & (83)\\
= & e^{-sT}\underbrace{\int_0^\infty h(t)e^{-st}dt}_{H(s)} & (84)\\
= & e^{-sT}H(s) & (85)
\end{aligned}$$

- Convolution:

$$f(t) * g(t) \Leftrightarrow F(s)G(s) \tag{86}$$

6.4 Filters

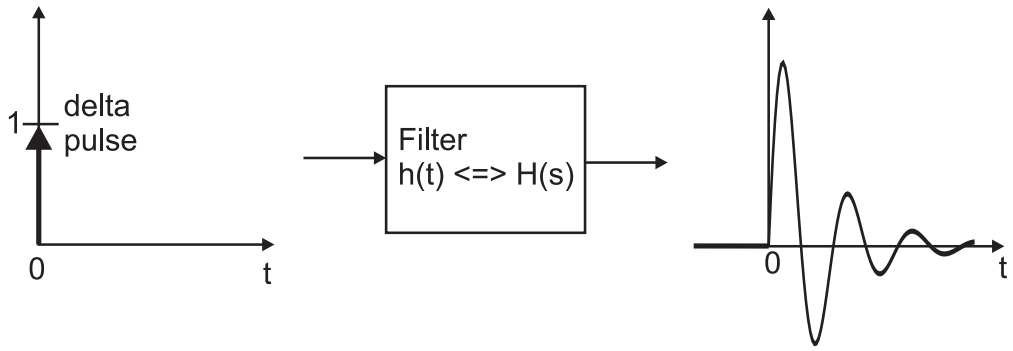


Figure 14 General idea of a filter and how to describe it: either with its impulse response or with it's Laplace transforms.

Fig. 14 presents a causal filter as a black box in continuous time. We send in a signal and and we get a signal out of it. The operation of the filter is that of a convolution of the input signal or a multiplication in the Laplace space:

$$g(t) = h(t) * x(t) \Leftrightarrow Y(s) = H(s) \cdot X(s) \tag{87}$$

How to characterise filters?

1. **Impulse Response**

$$x(t) = \delta(t) \quad \leftarrow \text{delta pulse} \tag{88}$$
$$h(t) = y(t) \quad \leftarrow \text{impulse response} \tag{89}$$

   The filter is fully characterised by its impulse response $h(t)$

2. **Transfer function** The Laplace transform of the impulse response is called *Transfer Function*. With the argument $j\omega$ we get the frequency response of the filter. What does the frequency response tell us about the filter? The absolute value of the

$$|H(i\omega)| \tag{90}$$

gives us the amplitude or magnitude for every frequency (compare the Fourier transform). The angle of the term $H(j\omega)$ gives us the phase shift:

$$\phi = \arg\left(H(i\omega)\right) \tag{91}$$

of the filter. In this context the *group delay* can be defined as:

$$\tau_\omega = -\frac{d\phi(\omega)}{d\omega} \tag{92}$$

which is delay for a certain frequency ω. In many applications this should be kept constant for all frequencies.

6.5 The z-transform

The Laplace transform is for *continuous* causal signals but in DSP we have *sampled* signals. So, we need to investigate what happens if we feed a sampled signal:

$$x(t) = \sum_{n=0}^{\infty} x(n)\delta(t - nT) \qquad \text{Sampled signal} \tag{93}$$

into the Laplace transform:

$$X(s) = \sum_{n=0}^{\infty} x(n)e^{-snT} \quad (94)$$

$$= \int_0^{\infty} x(n)e^{-st}dt \quad (95)$$

$$= \sum_{n=0}^{\infty} x(n) \underbrace{(e^{-sT})^n}_{z^{-1}=e^{-sT}} \quad (96)$$

$$= \sum_{n=0}^{\infty} x(n)(z^{-1})^n \quad \text{z-transform} \quad (97)$$

What is $e^{-sT} = z^{-1}$? It's a unit delay (see Eq. 80).

6.6 Frequency response of a sampled filter

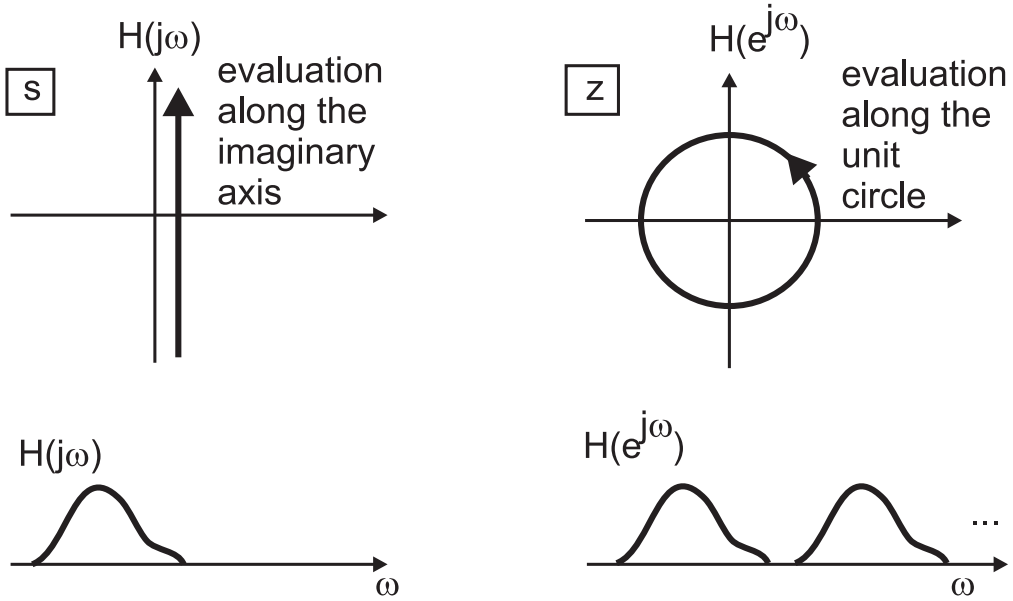


Figure 15 Comparing the calculation of the frequency response in the continuous case and sampled case.

Reminder: for analogue Signals we had $H(s)$ with $s = j\omega$ for the frequency response. Let's substitute s by z: $z^{-1} = e^{-sT}$ which gives us in general a mapping between the continuous domain and the sampled domain:

$$z = e^{sT} \tag{98}$$

With $s = j\omega$ this gives us $z = e^{j\omega}$ and therefore the frequency response of the digital filter is:

$$H(e^{j\omega}) \tag{99}$$

which then leads to the amplitude and phase response:

- Amplitude/Magnitude:

$$|H(e^{i\omega})| \tag{100}$$

- Phase

$$\arg H(e^{i\omega}) \tag{101}$$

Fig. 15 shows the difference between continuous and sampled signal. While in the continuous case the frequency response is evaluated along the imaginary axis, in the sampled case it happens along the unit circle which makes the response periodic! This is a subtle implication of the sampling theorem.

6.7 FIR Filter

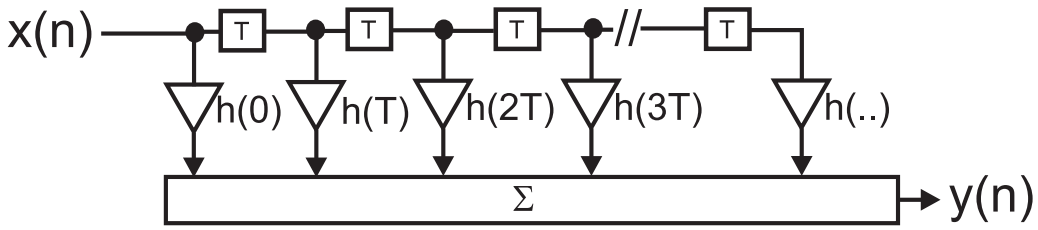


Figure 16 FIR filter using the impulse response of the analogue filter $h(t)$

What happens if we sample the impulse response $h(t)$ of an analogue filter? Let's find out:

$$h(t) = \sum_{n=0}^{\infty} h(nT)\delta(t - nT) \tag{102}$$

If we transform it to the Laplace space it looks like this:

$$H(s) = \sum_{n=0}^{\infty} h(nT) \underbrace{\left(e^{-sT}\right)^n}_{z^{-1}} \tag{103}$$

Remember that e^{-sT} has a very handy meaning: it is a delay by the unit time step (Eq. 80). Thus z^{-n} is a delay by n time steps. We can rewrite Eq. 103:

$$H(z) = \sum_{n=0}^{\infty} h(nT)(z^{-1})^n \tag{104}$$

This is the z-transform of the impulse response $h(t)$ of the filter.

We filter now the signal $X(z)$ with $H(z)$:

$$H(z)X(z) = \underbrace{\sum_{n=0}^{\infty} h(nT)z^{-n}}_{H(z)} X(z) \tag{105}$$

This sum is a direct recipe how to filter the signal $X(z)$. We only need the impulse response of the filter $h(nT)$ and we can set up a digital filter (see Fig. 16). Of course in practise this impulse response cannot run till infinity but only for a limited number of samples. These are often called “taps”. So for example a filter with 100 samples of $h(nT)$ has 100 “taps”. This in turn then requires a delay line which can hold $M = 100$ samples:

$$H(z)X(z) = \underbrace{\sum_{m=0}^{M} h(mT)z^{-m}}_{H(z)} X(z) \tag{106}$$

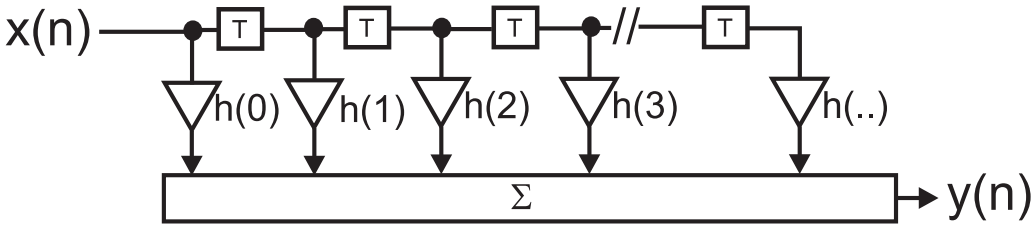


Figure 17 Digital FIR filter with normalised frequency

This is the formula of an Finite Impulse Response filter where we have sampled an analogue impulse response $h(t)$ at time intervals of T. However, usually the impulse response of the filter is directly derived in the digital domain where the argument n of $h(n)$ represents just the sample index n and the sampling interval is implicitly the inverse of the sampling rate. For that reason one needs to distinguish between:

$$H(z)X(z) = \sum_{m=0}^{M} h_{\text{analogue}}(mT)z^{-n}\,X(z) \quad (107)$$

$$H(z)X(z) = \sum_{m=0}^{M} h_{\text{digital}}(m)z^{-n}\,X(z) \quad (108)$$

$$= \sum_{m=0}^{M} h_{(}m)z^{-m}\,X(z) \quad (109)$$

Eq 107 is the filter where the impulse response is based on an analogue filter and Eq. 108 is based on the impulse response originating from a digital filter design (see section None and None) where the frequencies are normalised frequencies from 0 to 0.5. From now on we will always refer to the “digital” one (Eq. 108) and the subscript will be omitted (Eq. 109 & Fig. 17).

In the time domain the FIR filter is then doing the following operation:

$$y(n) = \sum_{m=0}^{M-1} h(m)x(n-m) \quad (110)$$

Generally FIR filters do not perform a convolution operation

Remember that we have derived the FIR filters from the analogue domain by filtering a signal (Eq.105). The timedomain representation of this is a discrete and causal convolution operation (using the sampled Laplace transform). However, the problem is that this runs to *infinite time* (see Eq 110) and would require an infinite number of delay steps and thus infinite time to complete. So the FIR filter at best performs an approximation of the convolution operation but there are serious problems with the finite number of taps (N) which requires a technique called windowing which moves it even further away from a convolution operation. On the other hand if the impulse response is strictly limited in time one can use this property for example for matched filters. However, the bottomline is that sweeping statements such as that “FIR filters perform a convolution” are *generally wrong* if nothing is known of the impulse response.

FIR filter implementations

No matter the design process the implementation is always the same: we need a delay line for the incoming signal and then weight the delayed outputs by the different coefficients and for that reason are called (water-) “taps” (see Eqs. 107 & 108). We are now presenting different implementations.

- C++: This is a simple example of a filter which stores the values in a simple linear buffer `bufferFIR` which stores the delayed values. The coefficients are stored in `coeffFIR`.

```
float filter(float value) {
  // shift
  for (int i=taps-1;i>0;i--)  {
    bufferFIR[i]=bufferFIR[i-l];
  }
  //store new value
  bufferFIR[0]=value;
```

```
    //calculate result
    for (int i=0;i<taps;i++)  {
      output +=bufferFIR[i]*coeffFIR[i];
    }
  return output;
  }
```

- Python: Here, the FIR filter is implemented as a class which receives the FIR filter coefficients in the constructor and then filters a signal sample by sample in the function filter:

```
class FIR_filter:
    def __init__(self,_coefficients):
        self.ntaps = len(_coefficients)
        self.coefficients = _coefficients
        self.buffer = np.zeros(self.ntaps)

    def filter(self,v):
        self.buffer = np.roll(self.buffer,1)
        self.buffer[0] = v
        return np.inner(self.buffer,self.coe
```

which again processes the signal sample by sample. It uses the numpy “roll” command to shift the samples and then the inner product to calculate the weighted sum between the buffer and the coefficients.

If one wants to filter a whole array one can use Python’s lfilter command:

```
import scipy.signal as signal
y = signal.lfilter(h,1,x)
```

This filters the signal x with the impulse response h. Note that this operation is on an array and thus a-causal.

More sophisticated code can be found in Teukolsky et al. (2007) . This book is strongly recommended for any C programmer who needs efficient solutions.

Fixed point FIR filters

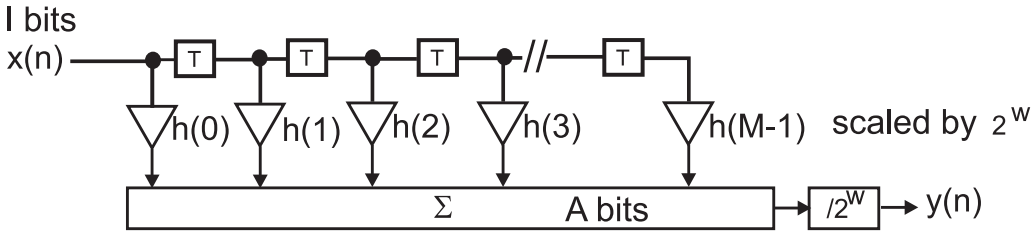


Figure 18 Fixed point FIR filter. The output signal is bit shifted to the right by w bits while the coefficients are scaled up by 2^w.

These filters receive integer numbers as input, perform integer multiplications/additions and their outputs are integer as well. Thus, these filters do not require a floating point unit on a processor.

Fig. 18 shows a fixed point FIR filter. The input $x(n)$ is an integer variable with I bit integers, the accumulator is an integer variable with A bits and the output as well (usually the same as the input in terms of bit width).

In contrast to a floating point FIR filter we need to scale up the coefficients so that they use full the integer range to avoid quantisation errors. For example if the coefficients of $h(n)$ range from -0.75 and $+0.75$ and we have signed 16 bit integers then the scaling factor is $2^W, W = 15$.

However, the accumulator A which collects the data needs to have more bits because it receives scaled input values at I bits precision and these multiplied by factor 2^W. If we have M taps then the additional bits we need is $\log_2 M$. The total number of bits we need in the accumulator in the worst case are:

$$A = I + W + \log_2 M \tag{111}$$

However, this is the worst case scenario because if the gain of the FIR filter is below one then the summations by the M taps will only create *temporary* overflows because integer numbers are cyclic in their representation. In case the gain of the FIR filter is below one this can be relaxed:

$$A = I + W \tag{112}$$

The actual bitwidth of the accumulator is usually the next integer size available and also makes sure that in case the gain goes slightly over one in an unexpected case that the filter still works. For example if I has 16 bits the accumulator has probably 32 bits.

Constant group delay or linear phase filter

So far the FIR filter has no constant group delay which is defined by:

$$\tau_\omega = -\frac{d\phi(\omega)}{d\omega} \tag{113}$$

This means that different frequencies arrive at the output of the filter earlier or later. This is not desirable. The group delay τ_ω should be constant for all frequencies ω so that all frequencies arrive at the same time at the output y of the filter.

A constant group delay can be achieved by restricting ourselves to the transfer function:

$$H(e^{i\omega}) = B(\omega)e^{-i\omega\tau+i\phi} \tag{114}$$

where $B(\omega)$ is real and the phase is only defined by the exponential. The phase of this transfer function is then trivially $\omega\tau + \phi$. The group delay is the derivative of this term which is constant.

Eq. 114 now imposes restrictions on the impulse response $h(t)$ of the filter. To show this we use the inverse Fourier transform of Eq. 114 to get the impulse response:

$$h(n) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} H(e^{i\omega}) e^{i\omega n} d\omega \tag{115}$$

After some transformations we get:

$$h(n+\tau) = \frac{1}{2\pi} e^{i\phi} b(n) \tag{116}$$

where $b(n)$ represents the Fourier coefficients of $B(\omega)$. Since B is real the coefficients $b(n)$ have the property $b(n) = b^*(-n)$. Thus we get a second impulse response:

$$h(n+\tau) = \frac{1}{2\pi} e^{i\phi} b^*(-n) \tag{117}$$

Now we can eliminate b by equating Eq. 116 and Eq. 117 which yields:

$$h(n+\tau) = e^{2i\phi} n^*(-n+\tau) \tag{118}$$

With the shift τ we have the chance to make the filter “more” causal. We can shift the impulse response in positive time to get $h(n)$ zero for $n<0$. In a practical application we shift the impulse response by half the number of delays. If we have a filter with M taps we have to delay by $\tau = M/2$.

The factor $e^{2i\phi}$ restricts the values of ϕ because the impulse response must be real. This gives us the final FIR design rule:

$$h(n + M/2) = (-1)^k h(-n + M/2) \qquad (119)$$

where $k = 0$ or 1. This means that the filter is either symmetric or antisymmetric and the impulse response has to be delayed by $\tau = M/2$.

Window functions

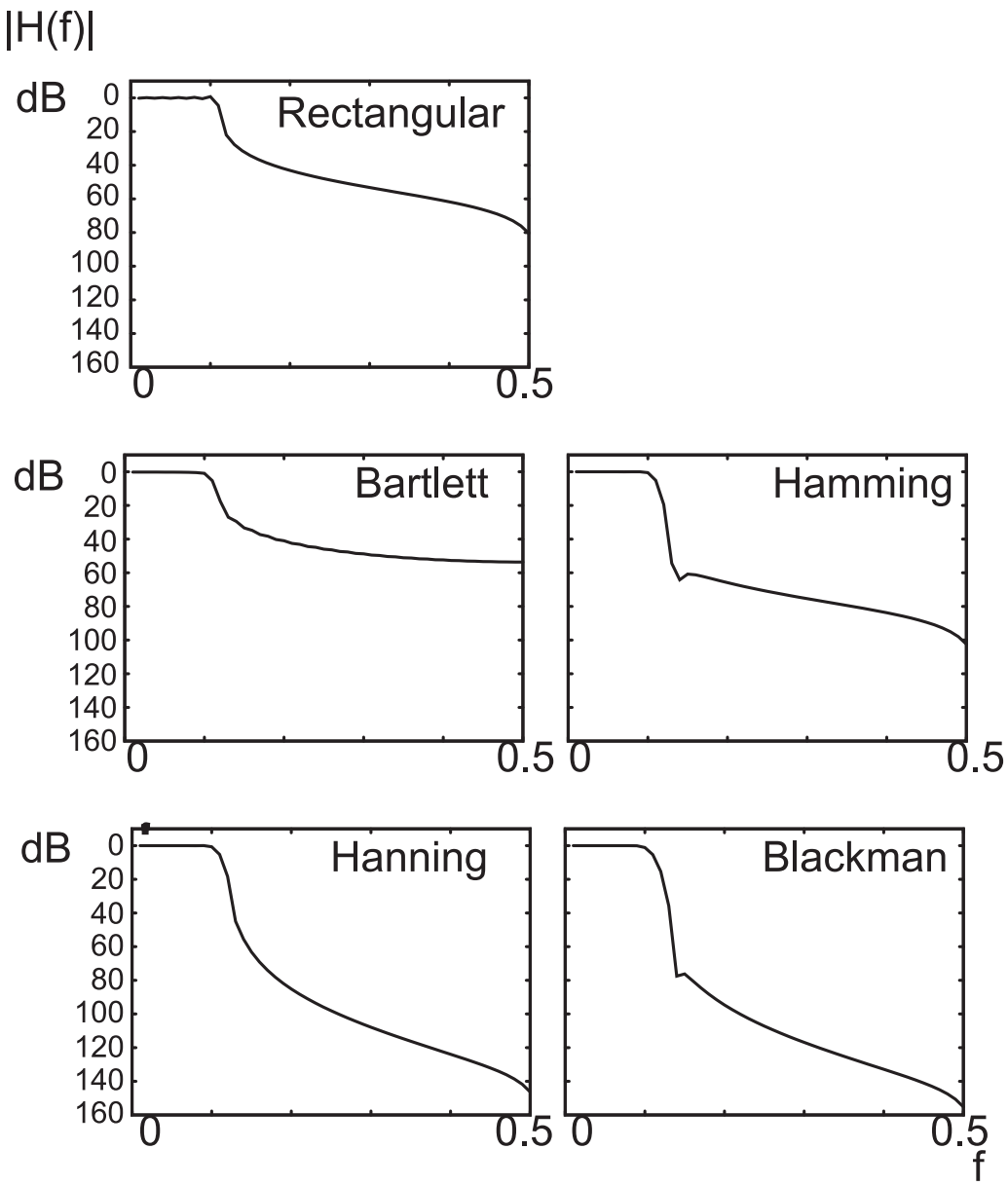


Figure 19 Different window functions applied to a low pass filter (Eq. 132) with cutoff at $f = 0.1$ and 100 taps.

So far we still have a infinite number of coefficients for for the FIR filter because there's no guarantee that the impulse response becomes zero after $M/2$ delays. Thus, we have to find a way to truncate the response without distorting the filter response.

The standard technique is to multiply the coefficients with a window function which becomes and stays zero at a certain coefficient $n > N$ so that Eq. 106 need not to run to infinity:

$$H(z)X(z) = \sum_{n=0}^{N} \underbrace{h(nT)w(nT)}\, z^{-n} X(z) \qquad (120)$$

1. Rectangular window: truncating the impulse response. Problem: we get ripples in the frequency-response. The stop-band damping is poor
2. Triangular or Bartlett window: greatly improved stop-band attenuation.
3. Hanning and Hamming window: standard windows in many applications.

$$w(n) = \alpha - (1-\alpha)\cos\left(\frac{2\pi n}{M}\right) \qquad (121)$$

   - Hamming: $\alpha = 0.54$
   - Hanning: $\alpha = 0.5$

4. Blackman window:

$$w(n) = 0.42 + 0.5\cos\left(\frac{2\pi n}{M}\right) + 0.08\cos\left(\frac{4\pi n}{M}\right.$$

5. Kaiser window: control over stop- and passband. No closed form equation available.

To illustrate how window functions influence the frequency response we have taken an impulse response of a lowpass filter ($f_c = 0.1$) and applied different window functions to it (Fig. 19).

Note that the higher the damping the wider the transition from pass- to stopband. This can be seen when comparing the Blackman window with the Hamming window (Fig. 19). For the lowpass filter this seems to be quite similar. However, for a bandstop filter the wider transition width might lead actually to very poor stopband damping. In such a case a Hamming window might be a better choice.

Python code: impulse response from the inverse DFT - The frequency sampling method

Imagine that we want to remove $50Hz$ from a signal with sampling rate of $1kHz$. We define an FIR filter with 100 taps. The midpoint $N/2 = 50$ corresponds to $500Hz$. The $50Hz$ correspond to index 5.

```
f_resp=np.ones(100)
# note we need to add "+1" to the end because th
# the range is not included.
f_resp[4:6+1]=0
f_resp[94:96+1]=0
hc=np.fft.ifft(f_resp)
h=np.real(hc)
# this is from index 0 to index 49 on the left
# and on the right hand side from index 50 to in
h_shift[0:50]=h[50:100]
h_shift[50:100]=h[0:50]
h_wind=h_shift*hamming(100)
```

To get a nice symmetric impulse response we need to shift the inverse around 50 samples.

FIR filter design from ideal frequency response – The analytical way

For many cases the impulse response can be calculated analytically. The idea is always the same: define a function with the ideal frequency response

$$|H(e^{j\omega})| = \underbrace{B(e^{j\omega})}_{\text{real}} \tag{123}$$

and perform an inverse Fourier transform to get the impulse response $h(n)$.

We demonstrate this with a lowpass filter:

$$|H(e^{j\omega})| = \begin{cases} 1 & \text{for } |\omega| \leq \omega_c \\ 0 & \text{for } \omega_c < |\omega| \leq \pi \end{cases} \tag{124}$$

Use the inverse Fourier transform to get the impulse response:

$$h(n) = \frac{1}{2\pi} \int_{-\pi}^{\pi} H(e^{j\omega}) e^{j\omega n} d\omega \tag{125}$$

$$= \frac{1}{2\pi} \int_{-\omega_c}^{+\omega_c} e^{j\omega n} d\omega \tag{126}$$

$$= \frac{1}{2\pi} \left[\frac{1}{jn} e^{j\omega n} \right]_{-\omega_c}^{+\omega_c} \tag{127}$$

$$= \frac{1}{2\pi jn} \left(e^{j\omega_c n} - e^{-j\omega_c n} \right) \tag{128}$$

With these handy equations:

$$\sin z = \frac{1}{2j}(e^{zj} - e^{-zj}) \tag{129}$$

$$\cos z = \frac{1}{2}(e^{zj} + e^{-zj}) \tag{130}$$

we get for the filter:

$$h(n) = \begin{cases} \frac{1}{\pi n}\sin \omega_c n & \text{for } n \neq 0 \\ \frac{\omega_c}{\pi} & \text{for } n = 0 \end{cases} \tag{131}$$

This response is a-causal! However we know that we can shift the response by any number of samples to make it causal! And we have to window the response to get rid of any remaining negative contribution and to improve the frequency response.

Highpass, bandstop and bandpass can be calculated in exactly the same way and lead to the following ideal filter characteristics:

- **Lowpass** with cutoff frequency $\omega_c = 2\pi f_c$:

$$h(n) = \begin{cases} \frac{\omega_c}{\pi} & \text{for } n = 0 \\ \frac{1}{\pi n}\sin(\omega_c n) & \text{for } n \neq 0 \end{cases} \tag{132}$$

- **Highpass** with the cutoff frequency $\omega_c = 2\pi f_c$:

$$h(n) = \begin{cases} 1 - \frac{\omega_c}{\pi} & \text{for } n = 0 \\ -\frac{1}{\pi n}\sin(\omega_c n) & \text{for } n \neq 0 \end{cases} \tag{133}$$

- **Bandpass** with the passband frequencies $\omega_{1,2} = 2\pi f_{1,2}$:

$$h(n) = \begin{cases} \frac{\omega_2 - \omega_1}{\pi} & \text{for } n = 0 \\ \frac{1}{\pi n}(\sin(\omega_2 n) - \sin(\omega_1 n)) & \text{for } n \neq 0 \end{cases}$$

- **Bandstop** with the notch frequencies $\omega_{1,2} = 2\pi f_{1,2}$:

$$h(n) = \begin{cases} 1 - \frac{\omega_2 - \omega_1}{\pi} & \text{for } n = 0 \\ \frac{1}{\pi n}(\sin(\omega_1 n) - \sin(\omega_2 n)) & \text{for } n \neq 0 \end{cases}$$

See Diniz (2002 , p.195) for more impulse responses.

Here is an example code for a bandstop filter which fills the array h with the analytically calculated impulse response:

```
f1 = 45.0/fs
f2 = 55.0/fs
n = np.arange(-200,200)
h = (1/(n*np.pi))*(np.sin(f1*2*np.pi*n)-np.sin(f
h[200] = 1-(f2*2*np.pi-f1*2*np.pi)/np.pi;
h = h * np.hamming(400)
```

After the function has been calculated it is windowed so that the cut off is smooth. It's not very elegant as it's causes a division by zero first and then the coeffecient at 200 is fixed. Do it better!

Design steps for FIR filters

Here are the design steps for an M tap FIR filter.

1. Get yourself an impulse response for your filter:

   (a) Create a frequency response "by hand" just by filling an array with the desired frequency response. Then, perform an inverse Fourier transform (see section None).

   (b) Define a frequency response analytically. Do an inverse Fourier transform (see section None).

(c) Use an analogue circuit, get its impulse response and use these as filter coefficients.

(d) Dream up directly an impulse response (for example, averagers, differentiators, etc)

2. Mirror the impulse response (if not already symmetrical)
3. Window the impulse response from an infinite number of samples to M samples.
4. Move the impulse response to positive time so that it becomes causal (move it $M/2$ steps to the right).

FIR filter design with Python's high level functions

The "firwin" command generates the impulse response of a filter with m taps and also applies a default window function. For example:

```
from scipy.signal import firwin
h = firwin(m,2*f)
```

generates a lowpass FIR filter with the normalised frequency f. Note the factor two because in scipy the normalisation is *not* the sampling rate but the *nyquist frequency*. To be compatible with the math here the easiest approach is to multiply the frequency by 2. With this command one can also design high pass, bandstop and bandpass filters. Type `help(firwin)` which provides examples for all filter types and which parameters need to be set.

Signal Detection: Matched filter

How can I detect a certain event in a signal? With a correlator. Definition of a correlator?

$$e(t) = \int_0^t \underbrace{s(\tau)}_{\text{signal}} \underbrace{r(\tau)}_{\text{template}} \, d(\tau) \tag{136}$$

How to build a correlator with a filter? Definition of filtering?

$$e(t) = \int_0^\infty s(\tau) h(t-\tau) d\tau \tag{137}$$

NB. h - integration runs backwards. However we used an int forward! $h(t) := r(T-\tau)$, only valid for $0 \dots T$.

$$e(t) = \int_0^T s(\tau) r\left(T - (t-\tau)\right) d\tau \tag{138}$$

$$= \int_0^T s(\tau) r(T-t+\tau) d\tau \tag{139}$$

for $t := T$ we get:

$$e(T) = \int_0^\infty s(\tau) r(\tau) d\tau \tag{140}$$

$$\underbrace{h(t) := r(T-t)}_{matched \quad filter!} \tag{141}$$

In order to design a detector we just create an impulse response h by reversing the template r in time and constructing an FIR filter with it.

How to improve the matching process? Square the output of the filter!

Adaptive FIR Least Mean Squares (LMS) filters

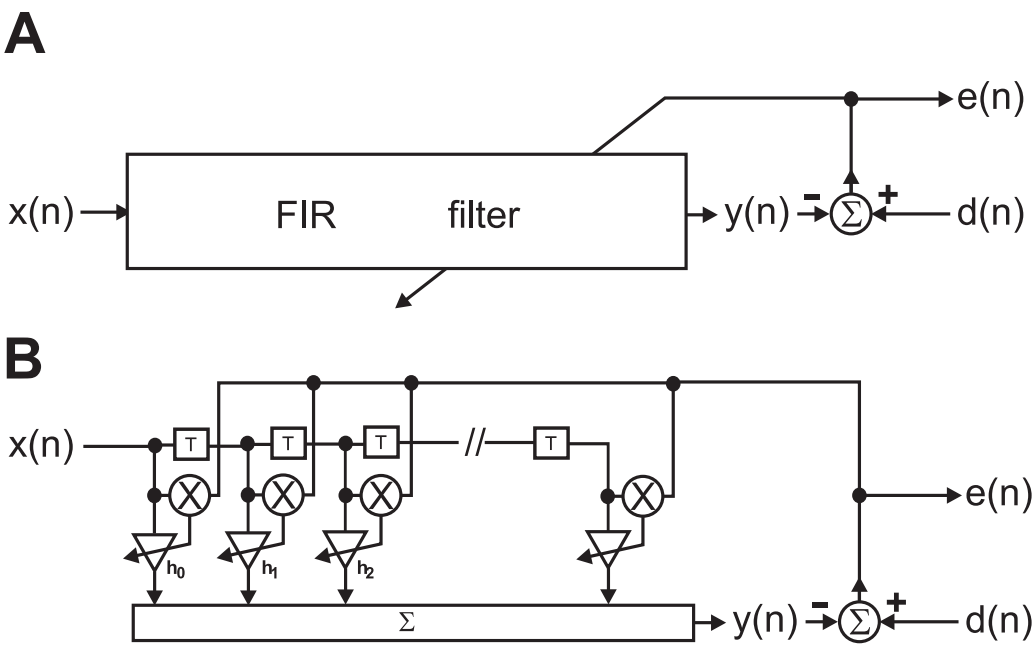


Figure 20 Adaptive FIR filter : A) Negative feedback loop where a desired signal $d(n)$ is compared with the output of the FIR filter $y(n)$ and creates the error signal $e(n)$ which then tunes the FIR filter. B) Internal workings of the adaptive FIR filter where the error signal $e(n)$ is multiplied with the delayed input signal $x(n-m)$ and then changes the FIR filter coefficients h_m.

So far the coefficients of the FIR filter have been constant but now we are going to allow them to change *while* the FIR filter is operating so that it can learn its own coefficients. The idea is to use an error signal $e(n)$ to tune the coefficients $h(n)$ of the FIR filter (see Fig. 20A) by comparing its output $y(n)$ with a desired output $d(n)$:

$$e(n) = d(n) - y(n) \tag{142}$$

and tuning the FIR filter in a negative feedback loop till the average of $e(n)$ is zero.

We need to derive how the error $e(n)$ can change the FIR filter coefficients so that the error is actually minimised. This can be expressed as a so called “gradient descent” which minimises the squared error $\frac{1}{2}e(n)^2$ because both a positive and a negative error are equally bad:

$$\Delta h_m = -\mu \frac{\partial \left(\frac{1}{2}e(n)^2\right)}{\partial h_m} \tag{143}$$

where $\Delta h_m(n)$ is the change of the coefficient $h_m(n)$ at every time step: $h_m(n+1) = h_m(n) + \Delta h_m(n)$. $\mu \ll 1$ defines how quickly the coefficients h_m change at every time step and is called the “learning rate”. Note here the change in notation of the FIR filter coefficients h_m which are changing much *slower* than the sampled signals $e(n), d(n)$ and $y(n)$ and can still be seen as constant for the realtime filtering operation. Thus, we have n for the sample by sample processing as before and the index m of h_m for the very slowly changing FIR filter coefficients. To gain some intuition why Eq. 143 mimimises the error $e(n)$ we look at what happens if we increase the FIR coefficient h_m a little bit, for example, by the small amount of $|\epsilon| \ll 1$: $h_m := h_m + \epsilon$. Then we can observe two cases:

1. The squared error $e(n)^2$ increases so we need to decrease h_m as it makes it worse.

2. The squared error $e(n)^2$ decreases so we need to increase h_m as it makes it better.